以 WebAssembly Component Model 實作日誌轉發框架

量化效能成本與可行性研究

Abstract

// 填入論文研究問題、解決方法、成效

壹、 緒論

1.1 研究背景

可觀測性（Observability）是指透過系統對外產生的訊號，掌握其內部運作狀態，並分析與解釋事件發生的原因，以協助後續的問題診斷及修復，因此在現代資訊系統的維運中具有重要地位。可觀測性通常由日誌（Logs）、追蹤（Traces）與指標（Metrics）三大支柱構成。其中，日誌能以直觀且具體的方式記錄系統在特定時間點所發生的事件，其內容通常包含時間戳記、日誌等級、訊息內容及追蹤識別碼（Trace ID）等資訊。這些資訊能提供事件發生時的詳細上下文，使維運人員得以依照事件順序追查系統行為，進而提高問題定位與處理的效率。

隨著系統規模及服務數量增加，應用程式所產生的日誌量也隨之成長。若日誌分散於不同主機、容器或服務中，不僅不利於統一查詢，也會增加問題排查的困難。因此，如何正確地蒐集、解析、過濾並轉送日誌至集中式後端，已成為可觀測性架構中的重要課題。在此過程中，日誌轉送器除了必須具備足夠的處理效能，也需要維持資料正確性與系統穩定性，盡可能避免日誌遺失、重複、毀損或亂序，以確保後續分析結果的可靠性。

本研究著眼於日誌轉送器的插件架構，運用 WebAssembly Component Model（WCM）所提供的標準化介面、跨語言互通及能力導向隔離等特性，設計並實作一套語言無關、處理階段可替換且故障影響範圍可控的日誌轉送框架。研究將透過吞吐量、延遲、資源使用量、資料正確性、突發流量恢復能力及長時間穩定性等一系列實驗，評估此架構的實際表現，並與現有日誌處理工具進行比較，以量化導入 WCM 所產生的效能成本，分析其優勢與限制，進而探討將 WCM 應用於日誌轉送系統的可行性。

1.2 研究動機

市面上現有的日誌轉發器，雖然各自具有成熟穩定的自訂邏輯方式，但普遍存在以下三個問題。第一項為語言綁定，在使用不同的轉發器時，若要插入自訂邏輯就必須使用該轉發器特定的開發語言，例如 Fluentd 便要使用 Ruby、Vector 則要使用自行設計優化的 VRL、Fluent Bit 的提供兩種開發方式，一種是 Lua 另一種則是 WASM 的插件，雖然 Fluent Bit 提供了同樣語言無關的 WASM 途徑，但仍然具有吞吐量遠低於 Lua 方式的弊病，結合以上各產品的特性，並無法自由的使用熟悉語言進行自訂邏輯開發。第二點是故障的隔離方式，根據實際實驗結

果，當插入外部邏輯時，若程式碼進入無窮迴圈、或長時間阻塞讀取檔案、OOM 等情況，會造成整條 workflow 的停滯，並無有效停止或發現問題的機制。第三點關於安全性，插入邏輯的開發通常沒有權限管理，可以任意向外存取網路、讀寫檔案，惡意程式碼甚至可能造成系統損壞。有鑑於以上三點問題，我思考能夠利用 WebAssembly Component Model 的特性，解決以上出現的問題，並提出一個日誌轉發框架與實作，且能完成基本的轉發器職責，與市面的產品進行實驗比較。

現有日誌轉送工具大多已具備成熟且穩定的擴充機制，使使用者能依需求加入解析、過濾、轉換與輸出等自訂邏輯。然而，這些擴充機制在開發語言、故障隔離及權限控管方面，仍存在值得探討的限制。

第一，不同工具的擴充機制通常與特定語言或執行環境緊密結合。例如，Fluentd 的插件主要以 Ruby 開發；Vector 多透過其領域特定語言 Vector Remap Language（VRL）實作資料轉換；Fluent Bit 則提供 Lua 與 WebAssembly（WASM）等過濾方式。當使用者更換日誌轉送工具時，往往需要重新學習相應的語言、介面與工具鏈，使既有程式邏輯不易直接移植。雖然 Fluent Bit 的 WASM 過濾機制提供了跨語言編譯的可能性，但在本研究的實驗條件下，其處理效能明顯低於 Lua 過濾方式。

第二，外部處理邏輯可能擴大系統的故障影響範圍。當插件發生無窮迴圈、長時間阻塞、記憶體持續成長、越界存取或執行期錯誤時，除了影響插件本身，也可能阻塞後續處理階段，使整條日誌處理管線停止輸出。專案中既有的故障注入與診斷資料也觀察到部分插件異常可能造成管線停滯。顯示確實存在故障無法重試或偵測的問題。

第三，傳統插件通常在宿主程序或受信任的執行環境中運作，插件可使用的系統資源未必能依處理階段進行細緻限制。若第三方插件能任意存取網路或檔案系統，一旦程式本身存在漏洞或包含惡意行為，便可能造成敏感資料外洩、檔案遭到修改，甚至影響宿主系統的穩定性。因此，日誌轉送框架除了需要提供擴充能力，也應該給予能力的限制，確保執行的安全。

基於上述問題，本研究採用 WebAssembly Component Model（WCM）設計並實作一套模組化日誌轉送框架。透過 WebAssembly Interface Types（WIT）定義標準化的插件介面，使不同程式語言所開發的元件能夠接入相同的 Parse、Filter、Format 與 Transport 處理管線；同時利用 WebAssembly 沙箱及 WASI 能力模型，依照各階段需求限制其可使用的系統資源，例如僅授予 Transport 元件網路存取能力。此外，本研究亦導入執行時間、記憶體與背壓控制機制，以縮小異常元件可能造成的影響範圍。並進一步透過吞吐量、延遲、CPU 與記憶體使用量、資料正確性、突發流量恢復能力及長時間穩定性等實驗，與 Fluentd、Vector、Fluent Bit Lua 及 Fluent Bit WASM 等現有方案進行比較，藉此量化 WCM 架構所產生的額外成本，並分析其在跨語言擴充、故障控制與權限隔離方面所帶來的效益，以評估 WCM 應用於日誌轉送系統的可行性。

1.3 研究問題

RQ1： WCM 與 WIT 在何種程度上能降低元件對特定程式語言與宿主介面的依賴，並限制元件的資源使用、權限及故障影響範圍？

RQ2： 相較於 Core Wasm 與現有日誌處理框架，WCM 在資料交換、吞吐量、延遲及資源使用上具有哪些成本與取捨？

RQ3： WCM 日誌處理系統在高負載、突發流量與長時間執行下，能否維持資料正確性、資源穩定性及過載後恢復能力？

貳、 背景知識與相關研究

2.1 集中化日誌蒐集

現代資訊系統通常由多種異質元件共同構成，例如應用程式、伺服器、資料庫及網路設備等。這些元件在運作過程中會持續產生日誌，用以記錄系統事件、執行狀態與錯誤資訊。隨著系統規模與服務數量增加，日誌的來源及資料量也隨之成長。若缺乏集中式管理機制，維運人員便需要分別登入不同主機或管理平台查詢日誌，不僅增加管理成本，也難以依照時間順序整合跨元件資訊，進而提高事件還原與故障排查的難度。

因此，現代可觀測性架構通常設置集中式後端，統一接收來自不同來源的日誌（Logs）、指標（Metrics）與分散式追蹤（Traces）。透過集中儲存、索引、查詢與關聯分析，維運人員能夠交叉比對不同的可觀測性訊號，釐清各元件在事件中的互動關係，進而縮短異常偵測、問題定位及故障排除所需的時間。

在此架構中，日誌轉送器（Log Forwarder）位於日誌來源與集中式後端之間，負責接收、處理並將日誌傳送至一個或多個目的端。當短時間內輸入大量日誌，且輸入速率超過轉送器或後端的處理能力時，系統可能出現佇列累積、傳送延遲上升、記憶體耗盡或日誌遺失等問題。因此，日誌轉送器的功能不僅是資料傳送，通常還包含日誌解析、資料過濾、格式轉換、批次處理、緩衝、路由、錯誤重試及背壓控制等機制，以降低突發流量造成的影響，並在受限資源與高負載條件下維持系統穩定性及資料正確性。

2.2 日誌轉發器之架構

日誌轉發器負責接收、處裡、傳送日誌，典型的日誌轉發器會採用 pipeline 的架構來進行，每個元件獨立執行一項任務，由多個元件串接成流水線完成對日誌的處理，管線化的處理有助於職責分離，一項元件只進行一個任務，方便元件的測試、替換。常見的處理包含解析、增富、過濾、路由、緩衝、傳輸等等，以下為典型的流程。

輸入 → 解析 → 過濾 → 序列化 → 傳送

2.3 WebAssembly Component Model

2.3.1 WebAssembly 特點與缺點

WebAssembly 是由四大瀏覽器公司共同研究的可攜式低階位元組碼(portable low-level bytecode)，目標是提供一個緊密表達、快速驗證編譯、且提供安全、近乎 0 開銷的執行效率。

WebAssembly 的特點有以下，第一項是語言、硬體、執行平台的獨立性，透過將不同程式語言都編譯至相同低階二進位碼，能保證運行環境不會影響運行的狀態。第二項是記憶體安全與隔離機制，透過強制規定的複製進出規定與和外部分開的獨立記憶體，保證 WebAssembly 檔案執行時的記憶體、程式碼空間、stack、內部資料結構與外部完全分離，即使內部產生錯誤或嘗試非法的記憶體跳轉均無法破壞這層防護確保安全性。第三點是快速啟動，

缺點的部分，第一點是 WebAssembly 對於匯入與匯出僅支援基礎型別(i32、i64、f32、f64)，在讀寫線性記憶體時需要自行利用 i32 的數字來線性記憶體定址位址，增加開發難度與出現錯誤的可能，並無支援高階型別，如 Array、Struct 等等。另一方面，模組間缺乏標準的互動介面，模組間的資料傳輸沒有固定序列化與讀取方式，如同上述沒有高階型別相同，只能依賴呼叫雙方事前的約定。

2.3.2 WebAssembly Component Model

為改善前述 WebAssembly 核心模組（Core WebAssembly Module）在跨語言互通、複合資料型別表達及模組組合方面的限制，WebAssembly 社群提出了 WebAssembly Component Model（以下簡稱 WCM）。WCM 是建構於核心 WebAssembly 之上的標準化元件架構，其主要目標是提升不同程式語言與函式庫之間的互通性，使各自獨立開發與編譯的 WebAssembly 模組，能夠透過明確定義的介面組合為可重複使用的軟體元件。

相較於核心 WebAssembly 僅提供整數與浮點數等低階資料型別，WCM 提供更具表達能力的元件層級型別系統，可描述字串（string）、串列（list）、紀錄（record）、變體（variant）及資源（resource）等高階資料型別。元件可透過 WebAssembly Interface Types（WIT）描述其匯入與匯出介面，使執行環境能在元件組合或實例化階段檢查介面的型別是否相容。此機制除了降低開發者手動處理資料格式與語言差異的負擔，也可避免因參數型別、回傳型別或介面定義不一致而產生的錯誤，從而提升元件間互動的型別安全性。

然而，核心 WebAssembly 無法直接處理上述高階型別，因此 WCM 進一步定義 Canonical ABI（Canonical Application Binary Interface），作為元件層級介面與核心 WebAssembly 函式之間的標準轉換規範。Canonical ABI 主要透過 lowering 與 lifting 兩種操作完成資料轉換：lowering 將元件層級的高階資料轉換為核心 WebAssembly 可處理的低階數值或線性記憶體表示；lifting 則進行相反的轉換，將核心 WebAssembly 回傳的低階資料重新解讀為元件層級的高階型別。

以字串為例，核心 WebAssembly 並不存在原生的字串型別。因此，Canonical ABI 在 lowering 過程中會依照指定的字元編碼，將字串內容寫入元件的線性記

憶體，並以記憶體位址和資料長度等低階數值傳入核心函式；函式執行完成後，lifting 操作則根據回傳的位址與長度讀取線性記憶體，驗證並解碼其中的資料，再將其還原為元件層級的字串。對於串列、紀錄及變體等複合型別，Canonical ABI 亦定義其欄位排列、記憶體配置、對齊方式及判別值的表示規則；必要時，呼叫端會透過元件提供的記憶體重新配置函式（例如 realloc）配置資料傳遞所需的空間。

藉由 Canonical ABI，不同語言的工具鏈只需將各自的語言型別對應至共同的元件型別及其標準底層表示，而不必為每一組語言分別設計呼叫協定。例如，Rust 與 C 對字串的內部表示方式雖不相同，但只要雙方皆遵循 Canonical ABI，即可透過相同的介面交換資料。換言之，Canonical ABI 將語言特定的資料表示方式與元件間的呼叫介面分離，為跨語言元件提供一致且可驗證的底層呼叫契約。

// 插入 lowering lifting 的流程
https://github.com/WebAssembly/component-model/blob/main/design/mvp/CanonicalABI.md
// 插入元件呼叫的流程圖

2.3.3 WebAssembly Interface Type

// https://component-model.bytecodealliance.org/design/wit.html
WebAssembly Interface Types（以下簡稱 WIT）是 WebAssembly Component Model 所使用的介面描述語言（Interface Description Language，IDL），用於定義元件之間的互動契約。WIT 不依賴特定程式語言，也不描述函式的實作方式或內部行為，而是以易於閱讀與撰寫的文字格式，描述元件匯入與匯出的介面、函式簽章及資料型別。透過語言中立的介面定義，不同程式語言實作的元件可以依循相同契約進行型別檢查、資料交換與組合，並可由工具鏈據此產生對應語言的繫結程式碼。因此，WIT 是建立、組合及實現跨語言 WebAssembly 元件互通的重要基礎。

2.3.4 WebAssembly System Interface

WebAssembly System Interface（以下簡稱 WASI）是一組標準化的系統介面，旨在使 WebAssembly 程式能以可攜且受控的方式存取執行環境所提供的外部功能，包括檔案與目錄操作、標準輸入輸出、時鐘、亂數及網路通訊等。WASI 介面通常由 WebAssembly Runtime 或宿主環境實作，再由 Runtime 將相關操作轉換為底層作業系統所提供的功能；因此，WASI 並非由作業系統直接提供給 WebAssembly 程式，而是在 WebAssembly 程式與宿主系統之間建立一層標準化抽象，使相同的程式不必直接依賴特定作業系統的 API。

自 WASI Preview 2 起，WASI 建立於 WebAssembly Component Model（WCM）之上，並採用 WebAssembly Interface Types（WIT）描述其標準介面。WIT 用於定義函式名稱、參數、回傳值及相關資料型別，並可透過 world 描述一個元件所匯入與匯出的完整介面集合。例如，元件可在 WIT 的 world 中匯入

WASI 所定義的檔案系統、亂數或時鐘介面，以宣告其執行時所需的外部能力。這類宣告僅描述元件的功能需求，實際功能仍須由 Runtime、宿主環境或其他相容元件提供。

2.4 現有的日誌轉發產品

2.4.1 Fluent bit 5.0.9

// https://docs.fluentbit.io/manual

Fluent Bit 是一套以 C 語言開發的開源遙測資料代理程式（telemetry agent），主要負責日誌（logs）、追蹤資料（traces）及指標（metrics）的接收、處理與轉送。根據官方文件，Fluent Bit 的設計目標是以較低的資源消耗處理高流量遙測資料，因此除了能部署於容器及雲端環境，也適用於資源受限的嵌入式系統與邊緣運算環境。

Fluent Bit 採用模組化的資料處理管線，其主要元件包含輸入（input）、解析器（parser）、處理器（processor）、過濾器（filter）、緩衝區（buffer）、路由器（router）及輸出（output）。遙測資料由 input plugin 接收後，會被轉換成 Fluent Bit 內部使用的結構化事件；事件可以先經由 parser 解析，再由 processor 或 filter 進行欄位轉換、資料補充、條件判斷或事件捨棄。處理完成的事件會進入緩衝與路由階段，最後由 output plugin 傳送至指定的後端系統。

資料處理流程

資料來源→Input→Parser／Processor→Filter→Buffer→Router→Output→目的地

Input plugin 產生事件後，Fluent Bit 引擎會將多筆事件組合成 chunk。輸入事件經解析與轉換後會被組成 chunk，後續再以 chunk 為單位進行排程、緩衝、路由、重試及輸出。根據官方文件，chunk 的平均大小約為 2 MB，但此數值並非可保證的固定批次大小，實際大小仍會受到輸入外掛及事件特性影響。

在執行模型方面，Fluent Bit 由主要事件迴圈協調資料收集、計時器、flush 排程與失敗重試。Input plugin 可選擇在獨立執行緒中執行，output plugin 則可配置一個或多個 worker 處理資料輸出；filter 通常由主要執行緒執行。因此，自訂 filter 的執行時間可能直接影響後續事件的處理進度。若 filter 發生長時間運算、阻塞或無窮迴圈，便可能阻礙整體資料管線，這也是本研究探討自訂邏輯故障隔離能力的重要原因之一。

自訂邏輯機制

本研究分別採用 Fluent Bit 的 Lua filter 與 WASM filter 實作相同的日誌過濾邏輯。Lua filter 允許使用者透過 Lua callback 讀取及修改事件，並以回傳值決定保留、修改或捨棄該筆事件。Fluent Bit 提供 protected_mode 選項，用於攔截部分 Lua 執行錯誤；然而，此機制主要處理語言執行期間的錯誤，並不等同於針對執行時間、CPU 使用量或外部資源存取所提供的完整隔離。Fluent Bit Lua Filter WASM filter 則允許使用者將以 Rust、TinyGo 等語言開發的邏輯編譯為

WebAssembly 模組，再由 Fluent Bit 載入執行。相較於 Lua，此方式降低了自訂邏輯對特定腳本語言的依賴。然而，Fluent Bit 的 WASM filter 採用其定義的函式介面與 C ABI，並不等同於 WebAssembly Component Model，也未直接使用 WCM 的介面型別、Canonical ABI 與元件組合機制。因此，本研究將 Fluent Bit WASM 視為傳統 WebAssembly 外掛方案，並與本研究提出的 WCM 架構比較其執行效能、語言互通性及故障隔離方式。

緩衝架構

當 Fluent Bit 的資料接收速率高於後端輸出速率時，尚未送出的 chunk 會先累積於緩衝區，形成背壓。Fluent Bit 支援記憶體緩衝、檔案系統緩衝及 Memory Ring Buffer 三種緩衝方式。記憶體緩衝具有較低的存取成本，但程序或主機異常時可能造成尚未送出資料遺失；檔案系統緩衝可提升資料持久性，但會增加磁碟 I/O；Memory Ring Buffer 則會在容量用盡時捨棄最舊的 chunk，以維持新事件的持續接收。由於本研究主要評估自訂過濾邏輯的執行成本，而非持久化緩衝能力，因此實驗沿用 Fluent Bit 的記憶體型 chunk 處理方式，未另外啟用檔案系統緩衝。如此可減少磁碟 I/O 對量測結果的干擾，使實驗更集中於 Lua、WASM 與本研究 WCM 架構之間的差異。綜合而言，Fluent Bit 的核心架構特色在於模組化資料管線、以 chunk 為基礎的緩衝與輸出機制、以 tag 和 match 為基礎的路由設計，以及可依工作負載調整的執行緒、worker 與緩衝模式。

2.4.2 Fluentd 1.16.3

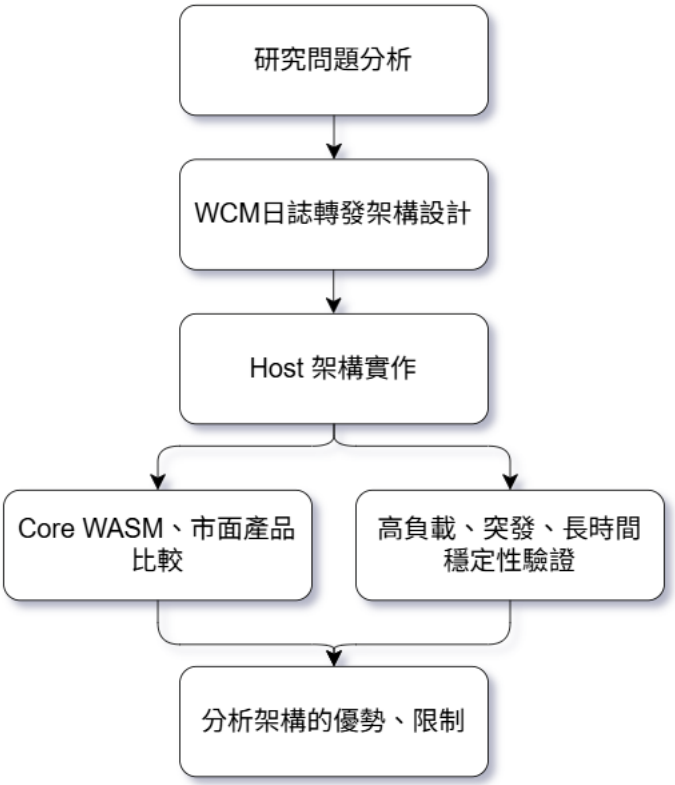
2.4.3 Vector 0.49.0

2.5 相關研究

參、 研究方法

研究架構

本研究採用設計科學研究法與控制實驗法。首先，針對傳統日誌處理插件依賴特定語言與產品介面、缺乏細粒度故障隔離，以及跨元件資料交換可能產生額外成本等問題，設計一套基於 WebAssembly Component Model 的日誌處理框架。其次，透過跨語言元件、故障注入、微型效能量測、產品比較、突發流量及長時間執行實驗，評估所提出架構的可攜性、隔離能力、執行成本與系統穩定性。整體研究流程如下：



//架構圖

3.1 WCM 與 WIT 在何種程度上能降低元件對特定程式語言與宿主介面的依賴，並限制元件的資源使用、權限及故障影響範圍？

現有日誌處理產品提供多種自訂邏輯擴充方式，包括使用產品本身的開發語言撰寫原生插件、使用 Lua 或 Ruby 腳本、載入 Wasm 模組，以及使用產品特化語言，例如 Vector Remap Language（VRL）。為了解不同擴充方式在異常情況下的行為，本研究先透過故障注入進行前導測試，並歸納出下列三項問題。

1. 當自訂邏輯出現非固定觸發的無窮迴圈時，部分 Lua 腳本與 Ruby 插件會持續占用執行緒，使整條處理管線停止輸出。若產品未提供插件層級的執行逾時機制，系統將無法主動中止異常邏輯，且可能缺乏明確的逾時警示。
2. 當插件的記憶體使用量持續增加時，部分方案只能依賴容器或作業系統提供的程序層級記憶體限制，無法單獨限制特定插件。當整體記憶體超過容器上限時，可能觸發 OOM，導致整個日誌處理程序或容器被強制中止。
3. 在本研究使用的產品配置與插件介面中，多數擴充方式未提供與 WebAssembly capability model 相當的細粒度能力授予機制。自訂邏輯可能使用執行環境已開放的檔案系統或環境變數能力，若要進一步限制，通常必須依賴容器、檔案權限或作業系統層級的隔離。

	Infinite loop	OOM	Data access
Fluent Bit Wasm	有	無	無
Fluent Bit Lua	有	有	有
Fluentd Ruby	有	有	有
Vector Lua	有	有	有

3.1.1 解決方法

針對上述問題本研究提出以契約驅動的元件化方法。首先，以 WIT 定義 Parsed Entry、Log Entry、Filter Result 及 Transport Config 等共用型別，再分別定義 Parser、Filter、Format 與 Transport 元件介面。Host 僅依賴 WIT 產生的 binding 呼叫元件，不直接依賴元件的來源語言、記憶體配置或函式 ABI。

透過上述 WIT 的機制，各種語言只需要透過各自的 wit-bindgen 工具產生對應的語言 binding 程式碼，再透過產生的程式碼物件撰寫各自邏輯，最後再編譯為相容的 WCM 元件即可替換，本研究分別使用 C、C#、Go 與 Rust 實作不同的處理階段，並由 Rust Host 統一載入與調度，以驗證不同語言元件能否在不修改 Host 呼叫流程的情況下完成替換。

3.2.2 資源、權限與故障隔離方法

資源與故障隔離並非單純由 WCM 的機制完成，而是由 Host 與 WCM 的邊界管理共同實現。本研究利用 Wasmtime 中實作的 ResourceLimiter 限制元件的記憶體用量，當元件要求的記憶體超過設定上限時，Wasmtime 將拒絕記憶體成長並產生 trap，以避免單一元件持續消耗 Host 記憶體。

針對無窮迴圈或長時間執行的元件，Host 使用 Wasmtime epoch interruption 設定最大執行時間。當元件超過期限時，執行階段將被中止並回傳錯誤，避免單一元件永久占用其所屬的處理執行緒。

在權限控制方面，本研究透過 WIT world、WASI import 與 Host Linker 共同決定元件可以使用的宿主能力。Parser、Filter 與 Format 元件不提供網路能力，只有 Transport 元件被授予 HTTP 介面。若元件匯入未被 Host 授權的網路或檔案系統介面，則會在實例化階段因找不到對應實作而被拒絕載入。當元件發生 trap、panic、逾時或記憶體超限時，Host 丟棄失敗實例、記錄錯誤，並嘗試處理後續批次。

3.2 相較於 Core Wasm 與現有日誌處理框架，WCM 在資料交換、吞吐量、延遲及資源使用上具有哪些成本與取捨？

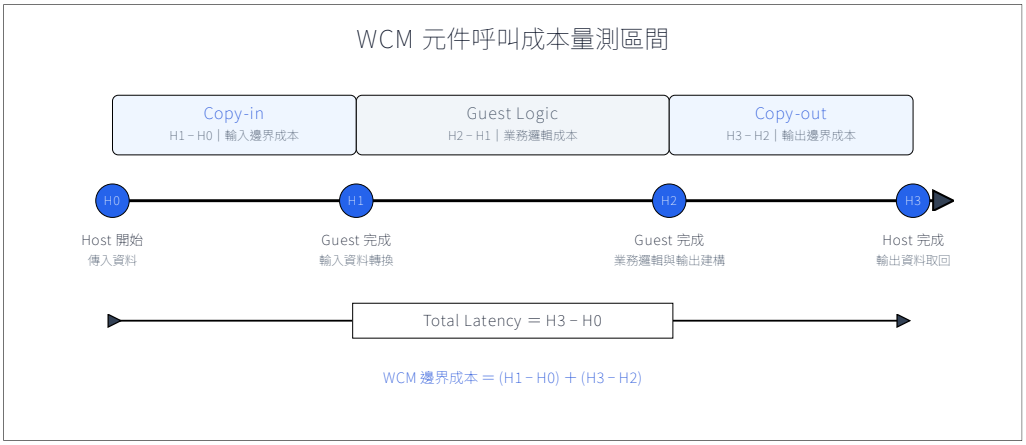
3.2.1 對照組選擇

為了區分 Core Wasm 與 WCM 的成本差異，本研究建立 Core Wasm with JSON

ABI 與 Core Wasm with Direct Memory ABI 兩個對照組。JSON ABI 代表容易實作，且具有語言中立性的通用介面；Direct Memory ABI 則代表開發者需要自行管理 offset、length、記憶體的低階介面，寫入 Guest 的線性記憶體，此方法能減少高階型別轉換，但開發者必須自行定義二進位格式、管理記憶體配置與釋放，並處理資料所有權及版本相容性，因此開發與維護成本較高。透過上述兩個對照組，可觀察 WCM 在「容易實作但序列化成本較高的 JSON ABI」與「效能較高但維護成本較高的 Direct Memory ABI」之間的相對位置。

3.2.2 如何分離邊界成本與業務邏輯

為避免將 JSON 解析或 Filter 邏輯誤認為 WCM 成本，本研究在 Host 與 Guest 中設置 H0 至 H3 四個時間點。H0 至 H1 代表輸入邊界成本，H1 至 H2 代表 Guest 業務邏輯，H2 至 H3 代表輸出邊界成本。三種 ABI 使用相同輸入資料與等價的處理邏輯，以比較 copy-in、guest logic、copy-out 與 total latency。



//測量示意圖

考量 H1 與 H2 的時間標記函式本身也需要由 Guest 呼叫 Host，再將控制權返回 Guest，若其執行時間占比過高，可能使邊界成本量測失真。因此，本研究另以空操作微型基準量測 time-sign 函式的呼叫成本。

量測結果顯示，單次完整 time-sign callback 的 p95 約為 47.575 ns，兩次呼叫合計約為 95.15 ns。在最小的非空批次中，兩次時間標記約占輸入與輸出邊界時間的 0.227%，因此其影響可視為量測誤差範圍，但仍將該數值記錄為系統性誤差上限。

3.2.3 市面產品的比較方法

本研究以固定 CPU、記憶體、輸入資料、Filter 規則、TCP 來源及 HTTP 輸出，分別執行本系統(WCM 架構)、Fluentd、Vector、Fluent Bit Lua 與 Fluent Bit WASM。實驗固定 CPU、記憶體、輸入資料、Filter 規則、TCP 來源及 HTTP 輸出，使各產品執行語意相近的 JSON 解析、條件過濾、格式化與傳送流程。主要比較指標包括：

- 最大可持續事件速率（EPS）
- 位元組吞吐量（MB/s）
- p50、p90、p95、p99 及 p99.9 延遲

- CPU 平均值與最大值
- 記憶體平均值與最大值
- 負載結束時的 backlog
- 資料完整性與 OOM 狀態

單純比較 EPS 可能忽略日誌大小、欄位數、批次策略與記憶體成本，因此本研究同時量測事件速率、位元組吞吐量、延遲與資源使用。此外，部分產品具有不同的內部 batch 與 flush 行為，因此結果用於分析整體產品取捨，不將所有差異直接歸因於 WCM。

3.3 WCM 日誌處理系統在高負載、突發流量與長時間執行下，能否維持資料正確性、資源穩定性及過載後恢復能力？

3.3.1 處理方式

為避免高負載下資料無限制累積，本研究在各處理階段之間使用有界 channel。當下游處理速度低於上游時，channel 填滿後會阻塞上游，使壓力逐步傳回輸入端。此方法以降低暫時吞吐量為代價，控制 Host 記憶體用量並避免因 queue 持續成長而發生 OOM。

3.3.2 驗證方式

使用三類實驗：

1. 正確性實驗：檢查遺失、重複、錯誤與亂序。
2. 突發實驗：流量由 baseline 提高至最大容量的 150%，持續 5、15、30 秒，再觀察恢復。
3. 長時間實驗：在最大容量 80% 下連續執行一小時。

觀察指標包括：

- 實際輸入與輸出速率
- p99 與 p99.9 延遲
- Output backlog
- Source pending
- CPU 與 RSS
- OOM 與重啟
- 恢復時間
- 資料正確性

對於非功能性的測試，上述三項測試涵蓋了日誌轉發運作過程中會出現的情況，首先是最重要的正確性實驗，確保轉發的過程中沒有遺失、重複，再來是短時間的突發流量與恢復，測試系統在短時間超出最大負載後仍能吸收並最終恢復正常，最後是長時間高負載，以 80%最大負載持續一小時，確保正確性。

肆、 系統設計與實作

本章說明本研究提出之 WebAssembly Component Model 日誌處理系統。系統的

主要目標，是在保留自訂日誌處理能力的同時，降低元件對特定程式語言及產品介面的依賴，並將元件的資源使用、系統權限與故障影響限制在可控制的範圍內。本研究採用契約驅動的元件化設計，以 WIT 定義 Host 與元件之間的資料型別及呼叫介面，並將處理流程拆分為 Parser、Filter、Format 與 Transport 四個階段。Rust Host 負責資料輸入、批次建立、元件調度、資源限制、背壓、路由與錯誤復原；各 WCM 元件則僅負責特定階段的業務邏輯。

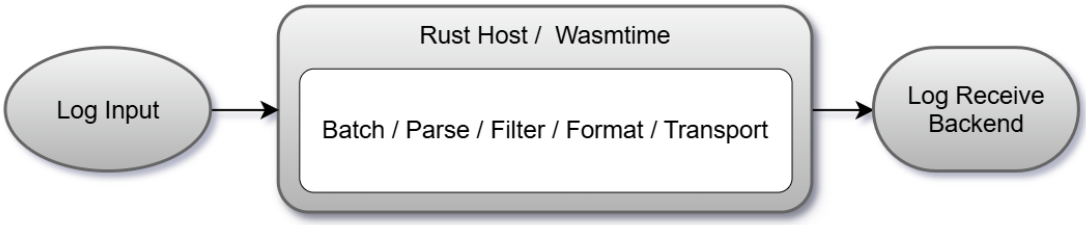
本章首先說明系統設計目標、系統假設與故障模型；其次介紹 WIT 契約設計及元件隔離方式；接著說明整體架構與資料流程；最後討論背壓、資源限制、故障復原及元件熱抽換等關鍵機制與設計取捨。

4.1 問題背景與設計目標

如第二章與第三章所述，現有日誌處理框架的自訂邏輯主要面臨三項問題：其一，自訂元件通常依賴特定程式語言、產品 API 或執行環境，增加跨系統移植成本；其二，元件可使用的記憶體、執行時間及系統能力缺乏明確邊界；其三，元件發生無窮迴圈、記憶體超限或執行錯誤時，故障可能擴散至整體處理流程。基於上述問題，本研究將語言中立性、資源與能力隔離、故障侷限及負載穩定性列為系統主要設計需求，並以解決上述問題為設計目標。

4.2 系統模型

本系統處理以行為單位的日誌資料，輸入來源包含持續讀取檔案、TCP 連線。日誌經過 Parser、Filter、Format、Transport 四個階段的處理後，轉送至一個或多個 HTTP 日誌接收端。



// 系統架構概要圖

系統包含以下兩項執行實體：

- Host：受信任的控制端，以 Rust 實作，負責資料與元件生命週期管理。
- Guest Component：執行自訂處理邏輯的 WCM 元件，可能由 C、C#、Go 或任何 wit-bindgen 與 WCM 支援的語言實作。

本研究考慮以下元件故障：

1. 元件回傳格式錯誤或處理錯誤。
2. 元件執行期間發生 Wasm trap 或 panic。
3. 元件進入無窮迴圈或執行時間過長。
4. 元件要求超過限制的線性記憶體。
5. Transport 元件遭遇連線失敗、逾時或目的端無回應。

以上主要處理元件層級與流水線層級的故障，不處理 Host 程序本身的程式錯誤、作業系統崩潰及實體機器故障。這些問題需要程序監控、容器重新排程或分散式容錯機制，超出本研究範圍。

4.2 WIT 契約與元件介面設計

4.2.1 介面設計原則

本系統以 WIT 作為 Rust Host 與 WCM 元件之間的唯一介面契約。WIT 負責描述元件的輸入輸出型別、錯誤型別及所需的 WASI 能力，不限定元件使用的實作語言。不同語言撰寫的元件只要符合相同 WIT 契約，即可由 Host 使用相同方式載入與呼叫。

整體資料流程保留結構化資料至 Format 階段。在 Parser、Filter 與 Format 之間，系統使用具有明確欄位的日誌型別交換資料；完成格式化後，資料才轉換為不透明的位元組，交由 Transport 傳送。此設計使前段元件能依欄位執行處理，同時使 Transport 不需要理解實際日誌格式。

WIT 契約同時劃分 Host 與元件的責任。日誌 ID、路由資訊、批次管理與資源控制由 Host 掌握；元件則只負責特定階段的業務邏輯。藉此避免元件任意修改系統識別資訊或直接控制其他處理階段。

4.2.2 共用資料型別

本系統主要使用 parsed-entry 與 log-entry 兩種日誌型別。parsed-entry 為 Parser 產生的解析結果，包含時間、日誌層級、訊息、標籤及路由標籤，但不包含資料 ID。Host 收到解析結果後，為每筆資料分配 ID，再轉換為後續階段使用的 log-entry。

將兩種型別分開的主要原因，是將資料解析與系統識別資訊分離。Parser 只負責理解原始日誌格式，不負責建立系統內部識別碼；日誌 ID 則統一由 Host 建立，使不同語言實作的 Parser 不需要自行協調 ID 產生方式。

路由標籤不包含於後續使用的 log-entry 中，而是由 Host 另外保存，並與日誌資料同步傳遞。此設計可避免 Filter 或 Format 元件修改日誌的目的端，使路由決策仍由 Host 控制。

4.2.3 Parser 元件介面

Parser 元件負責將原始日誌字串轉換為結構化資料。其輸入為一批原始字串，輸出為一批 parsed-entry。採用批次介面而非逐筆呼叫，可降低 Host 與元件之間的呼叫次數，但單次呼叫需要交換較多資料，因此批次大小仍須受到記憶體上限控制。

Parser 的輸出包含時間、日誌層級、訊息、標籤與路由標籤，但不包含 ID。Host 取得結果後，才為每筆資料建立 ID，並將路由標籤與日誌本體分開保存。此設計使 Parser 無法控制系統識別資訊，也能確保後續 Filter 回傳的決策可以由 Host 配對至原始資料。

Parser 使用獨立的解析錯誤型別，以區分格式錯誤、不支援的版本及資料毀損。這類錯誤表示元件已正常完成呼叫，但無法解析輸入資料；與無窮迴圈、記憶體超限或 Wasm trap 等執行環境錯誤不同，後者由 Host 的錯誤處理機制負責。

在能力設計上，Parser 僅需要接收 Host 傳入的資料，不需要自行讀取檔案或建立網路連線。因此，Parser 的 WIT World 不宣告檔案系統與 Socket 能力，使具有額外未授權 Import 的元件無法完成實例化。

4.2.4 Filter 元件介面

Filter 元件負責判斷每筆日誌是否應繼續進入後續流程。其輸入為一批 log-entry，輸出則是一批 filter-result。每項結果只包含日誌 ID 與布林型態的保留決策，不回傳修改後的完整日誌。

Filter 採用「回傳決策」而非「回傳資料」的設計，使原始日誌仍由 Host 保存與管理。Host 根據 Filter 回傳的 ID 找到對應資料，並依保留決策建立下一階段的批次。如此可避免 Filter 非預期修改時間、訊息、標籤或路由資訊，也可減少元件回傳完整日誌所需的資料交換量。

介面契約應要求 Filter 僅能回傳輸入批次中存在的 ID，且每個 ID 最多對應一項決策。若元件回傳不存在的 ID，應視為介面違反；未被保留的日誌則不再進入 Format 階段。

Filter 使用共用的元件錯誤型別，表示輸入錯誤、處理失敗、設定錯誤或內部錯誤。元件主動回傳的錯誤由 Host 接收並進行重試；若發生 trap 或執行逾時，則由 Wasmtime 執行控制機制中止呼叫。

4.2.5 Format 元件介面

Format 元件負責將結構化的 log-entry 轉換為可傳輸的輸出格式，例如 JSON、NDJSON、Syslog 或其他自訂格式。其輸入仍為一批結構化日誌，輸出則為位元組資料，使 Host 與 Transport 不需要理解特定輸出格式。

使用位元組作為輸出可避免介面限定為文字格式，也能支援未來的二進位編碼。然而，目前 WIT 僅將整批結果表示為單一位元組序列，因此系統另外使用長度前綴區分批次中的各筆輸出。每筆資料前方包含固定長度的資料大小欄位，Host 擷取長度後再還原為多筆格式化日誌。

Format 完成後，Host 將各筆輸出與原先保存的路由標籤重新配對。因此，介面契約必須要求 Format 的輸出筆數與輸入筆數一致，並維持相同順序，否則 Host 將無法判斷格式化結果應傳送至哪一個 Endpoint。

Format 不負責傳輸資料，也不需要知道後端 URL。將格式編碼與網路傳輸分離，可以獨立替換輸出格式或傳輸方式，避免 Format 元件與特定後端服務耦合。

4.2.6 Transport 元件介面

Transport 元件負責將格式化後的位元組資料傳送至外部日誌接收端。其介面分為初始化與傳輸兩個操作。初始化操作由 Host 提供端點、驗證方式、連線逾時、請求逾時、重試、TLS 及自訂標頭等設定；傳輸操作則接收多筆已格式化的位元組資料。

將初始化與傳輸分開，可使 Transport 元件在多次傳輸之間保留端點設定或連線相關狀態，避免每次呼叫都重新建立完整設定。當 Transport Worker 切換至不同 Endpoint 時，Host 會重新執行初始化，使同一類型的 Transport 元件能服務不同的目的端。

Transport 的輸入是多筆獨立的位元組資料，每筆資料不再包含日誌欄位資訊。Transport 只需負責組成請求並完成傳送，不需要重新解析日誌內容，因此可與 JSON、Syslog 或其他 Format 元件搭配使用。

Transport 是流水線中唯一需要對外網路能力的元件。Host 雖提供 WASI HTTP 或 Socket 介面，但仍會依據設定檔中的 Endpoint 清單檢查目的位址。即使 Transport 元件具有網路能力，也只能連線至已授權的後端服務。

4.2.7 錯誤契約設計

本系統將錯誤分為介面層錯誤與執行層錯誤。介面層錯誤由元件透過 WIT 結果型別主動回傳，表示元件已完成呼叫，但輸入資料、設定或業務邏輯發生問題。執行層錯誤則包括 Wasm trap、記憶體超限及執行逾時，由 Wasmtime 回報給 Host。

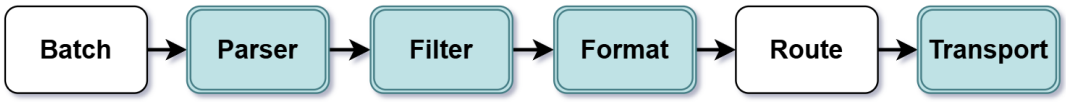
將兩類錯誤分開，可使 Host 根據錯誤性質採取不同處理方式。對可預期的處理錯誤，Host 可記錄原因或執行有限次重試；對 trap 或逾時，Host 則可將可能已進入異常狀態的元件實例丟棄並重新建立。

4.3 系統整體架構與處理流程

4.3.1 整體架構

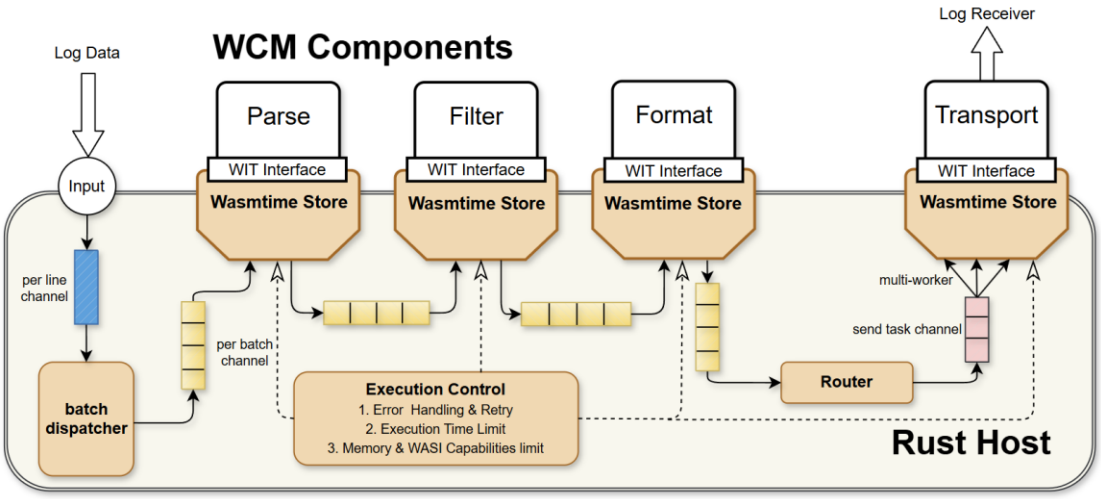
系統分為日誌資料處理與控制兩個面向。

日誌資料處理代表日誌的過程會需要經過哪些 pipeline 操作，如下圖所示包含 Batch、Parse、Filter、Format、Route、Transport，當中藍底雙框線的部分為 WCM 可替換的元件，能夠使用 WCM 有支援的任意語言開發替換，白底的為 Rust Host 負責完成。



控制則代表系統如何正確的將資料傳遞至下個階段，當中發生錯誤該如何處理，如下列。

- 初始化系統與載入設定。
- 建立 Wasmtime Engine、Store 及元件實例。
- 設定記憶體與執行時間限制。
- 管理元件版本與熱抽換。
- 處理錯誤、重試及實例重建。



// 系統總圖

4.3.2 日誌資料處理流程

本系統之資料處理流程如圖 4-X 所示。系統接收到以行(line)為單位的日誌後，會先將資料暫存於 Per-Line Channel，再交由 Batch Dispatcher 進行批次管理。Batch Dispatcher 設有三種批次送出條件：第一，批次累積資料量達到設定上限；第二，批次等待時間達到設定上限；第三，上游 Channel 關閉。其中，前兩項門檻可透過設定檔調整，第三項則通常發生於輸入來源結束或系統停止時。當任一條件成立後，Batch Dispatcher 會將目前累積的日誌封裝為一個 Batch，並送入 Per-Batch Channel 等待後續處理。

Parser、Filter、Format 與 Transport 在資料流上採用相似的處理模式，即從上游 Channel 取得資料、呼叫對應的 WCM 元件，並將處理結果送往下一階段。以 Parser 為例，當 Parser Worker 從 Per-Batch Channel 取得一個 Batch 後，會取得可用的 Parser Component Instance，透過 WIT 介面將批次資料傳入元件進行解析。Host 取回執行結果後，依結果進行必要的錯誤處理與重試；處理成功的結果則封裝為下一階段所需的批次資料，並送入下游 Channel。各階段的元件介面、資料型別及實例生命週期雖有所差異，但整體均遵循上述流水線模式，且可替換為其他符合相同 WIT 契約的元件實作。

流水線中較為特殊的處理包括 Filter 結果套用與 Transport 前的 Router。Filter 元件完成判斷後，會針對每筆日誌回傳 ID 與布林型態的保留決策，Rust Host 再依據該結果保留或移除對應日誌。另一方面，Parser 所產生的 targettag 會由 Host 保存，並隨日誌資料在後續階段中傳遞。格式化完成後，Router 根據 targettag 與設定檔中的 Endpoint 對應關係，將日誌依目的端分組並建立 SendTask，再送入 SendTask Channel 等待 Transport Worker 處理。若同一筆日誌指定多個目的端，Router 會執行 fan-out，將該日誌分別加入不同 Endpoint 的傳輸任務中。

4.3.3 控制說明

本系統的控制機制是指 Rust Host 對各項 WCM 元件進行載入、整合、調度、資源限制、版本管理與故障處理。元件僅負責特定階段的業務邏輯，其執行環境、系統權限與生命週期則由 Host 統一管理，以避免單一元件直接影響其他處理階段。

系統初始化時，Host 會分別為 Parser、Filter、Format 與 Transport 建立元件執行插槽。每個插槽保存 Wasmtime Engine、已編譯的 Component、Linker 及元件版本。Engine 負責編譯與執行 WCM 元件，Linker 則負責連結元件所需的 WIT 與 WASI 介面。一般資料處理元件採用同步執行環境；Transport 因涉及 HTTP 請求，則使用支援非同步 I/O 的執行環境。實際執行元件時，Host 會依據插槽中的 Engine、Component 與 Linker 建立 Wasmtime Store，並在 Store 中配置 WASI Context、資源表及記憶體限制器，最後建立對應的元件實例。不同階段依其執行特性採用不同的實例生命週期。四個階段皆使用預先建立的實例池，當有實例被取用果後會馬上補齊，減少當中的等待時間，每個實例使用過後即丟棄，設計原因會在 4.4.2 說明。

每個 Wasmtime Store 均具有獨立的線性**記憶體限制**。當元件要求擴張記憶體時，Host 會檢查目標大小是否超過設定上限；超出上限的成長要求將被拒絕。此限制可避免單一元件持續增加線性記憶體，進而耗盡整體系統資源。元件**執行時間限制**則透過 Wasmtime 的 Epoch Interruption 機制控制。Host 週期性更新 Engine 的 Epoch，並在每次呼叫元件前設定執行期限。若元件因無窮迴圈或其他異常狀況未能在期限內返回，Wasmtime 會中止該次執行並產生 trap，使 Host 能重新取得控制權。

除記憶體與執行時間外，Host 亦透過 WIT、Linker 與 WASI Context 限制元件可使用的系統能力。一般資料處理元件不會取得可存取的檔案目錄或對外網路位址；Transport 因負責資料傳送而具有網路能力，但其連線目的地仍受到 Endpoint 白名單限制，只能存取設定檔中明確列出的後端服務。

為支援執行期間的**元件更新**，系統設置 Config Watcher，定期檢查設定檔、元件路徑及元件檔案的修改時間。當偵測到元件變更時，Host 會先編譯並驗證新元件。只有在新元件成功建立後，才會更新對應的執行插槽與版本號。各處理階段在批次之間檢查元件版本，若發現版本變更，便丟棄舊實例並使用新元件建立執行環境。若新元件載入失敗，系統則保留原有版本繼續運作。

元件執行結果可分為正常結果、元件主動回傳的處理錯誤，以及 Wasmtime 產生的 trap、記憶體超限或執行逾時。發生錯誤後，Host 會依處理階段執行有限次重試，並在必要時丟棄可能已進入異常狀態的元件實例。若重試後仍無法完成處

理，Host 會記錄失敗原因及對應資料，停止該批資料進入後續階段，並繼續處理下一個批次，以將故障影響限制在單一元件實例與單一批次之內。

4.3.4 並行說明

本系統採用階段式流水線並行模型，使 Parser、Filter、Format 與 Transport 不必在同一執行序中依序完成。主執行序負責初始化系統、建立各處理階段與 Channel，並在系統結束時回收執行結果；各處理階段則具有獨立的執行環境與 Wasmtime Store。

實際執行架構具有進一步的階層劃分。Parser 階段包含負責建立批次的 Dispatcher 與負責呼叫元件的 Worker；Filter 與 Format 分別由獨立執行序處理；Transport 階段則包含 Router 與多個 Transport Worker。多個 Transport Worker 共用 SendTask Channel，但各自持有獨立的 Wasmtime Store 與元件實例，因此能同時執行不同 Endpoint 的網路傳輸。

各階段之間以 Channel 傳遞處理結果。當某一批資料進入 Filter 時，Parser 可同時處理下一批資料，而 Format 與 Transport 亦可處理更早送入的批次。此方式使不同批次能同時位於流水線的不同階段，減少網路等待或元件處理時間造成的整體閒置。系統中的 Channel 均具有固定容量。當下游處理速度低於上游時，尚未處理的批次會暫存於 Channel；Channel 容量用盡後，上游的傳送操作將被阻塞，使壓力逐步回傳至 Input Reader。此機制除了協調各階段的執行速度，也能限制流水線中的在途資料量，避免資料持續累積而耗盡 Host 記憶體。

元件呼叫所產生的處理錯誤、trap 或執行逾時會由所屬階段的 Host 邏輯攔截，並在該執行序內進行重試或實例重建。因此，單次元件呼叫失敗不會直接終止其他階段，其他執行序仍可繼續處理既有資料。然而，此隔離主要針對元件執行失敗；若階段執行序本身發生未處理的 panic 或意外終止，Channel 將隨之關閉，並可能使整條流水線逐步停止。Parser、Filter 與 Format 目前主要依批次順序進行處理，因此在這些階段中可維持批次的相對順序。Transport 則使用多個 Worker 平行傳送，不同 SendTask 的完成時間可能因 Endpoint 狀態與網路延遲而不同，因此系統不保證不同傳輸任務之間具有全域完成順序。

4.4 設計取捨

4.4.1 批次與單筆處理得比較

4.4.2 元件是否重用

伍、 系統評估與實驗

陸、 結論與未來工作

，結果顯示不同語言的元件均能被正確載入，輸入與輸出正確的格式，代表 WIT 能確實降低語言的耦合問題。

// 劣勢；階段越多速度越慢

柒、　參考資料